AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
i-CREDIT HOURS ENGINEERING PROGRAMS
Computer Engineering and Software Systems

CSE485–CSE488: Deep Learning

Assignment 1: MicroGrad

A Scalar Automatic Differentiation Engine

Documentation Report

Anas Tamer
Student ID: 22u0005

November 3, 2026

Contents

1 Introduction **2**

2 Assignment Objectives **2**

3 System Architecture **2**

4 The Computational Graph **2**

5 Value Class Design **3**

6 Operator Overloading **3**

7 Mathematical Derivations: Local Derivatives **4**

8 The Chain Rule **5**

9 Topological Sorting **5**

10 Backpropagation Algorithm **6**

11 Activation Functions **7**

12 Neural Network Implementation **7**
12.1 Neuron 7
12.2 Layer 7
12.3 MLP 8

13 Testing Methodology **8**

14 Testing Results **8**

15 Notebook Demonstration **9**

16 Design Justifications **10**

17 Conclusion **11**

1 Introduction

Modern deep-learning frameworks compute gradients automatically. This assignment builds that machinery from scratch at the smallest possible scale: every scalar is an object that remembers how it was produced, and gradients are obtained by applying the chain rule backwards through the resulting graph. The design follows Andrej Karpathy's `micrograd`.

The implementation consists of two files of the provided template:

- `src/engine.py`: the `Value` class (graph construction, local derivatives, topological sort, backpropagation);
- `src/nn.py`: `Neuron`, `Layer` and `MLP`, built only from `Value` objects.

No PyTorch, TensorFlow or JAX is used. The demo notebook uses NumPy, scikit-learn and Matplotlib only to generate data and draw plots.

2 Assignment Objectives

Following the assignment specification:

1. Implement `Value` to store scalar data and gradients.
2. Overload `__add__`, `__mul__`, `__pow__` (and the derived operators) to build the forward graph.
3. Implement local derivatives (`_backward`) for every operator and for `tanh` and `exp`.
4. Implement the global `backward()` using a topological sort.
5. Implement `Neuron`, `Layer` and `MLP` with random initialisation and forward passes.

3 System Architecture

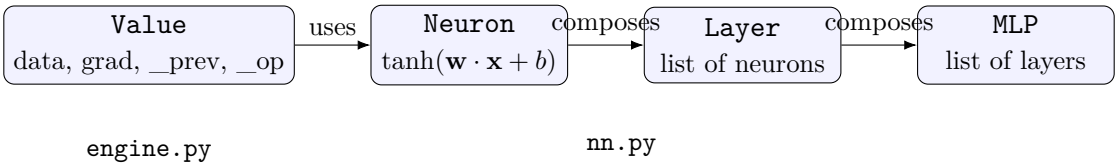


Figure 1: The two-layer architecture: the autodiff engine and the network API built on top of it.

The network classes never compute gradients themselves. They only create `Value` parameters and combine them with ordinary Python operators, and the engine does the rest. Each class exposes `parameters()` (a flat list of `Value` objects) and `zero_grad()`.

4 The Computational Graph

A computation such as $L = (a \cdot b + c) \cdot f$ is represented as a directed acyclic graph (DAG). Leaves are inputs or parameters, internal nodes are results of operations, and edges point from an operand to the result. Figure 2 shows this graph as produced by our engine (drawn in the notebook from `_get_topo()`) *after* calling `L.backward()`.

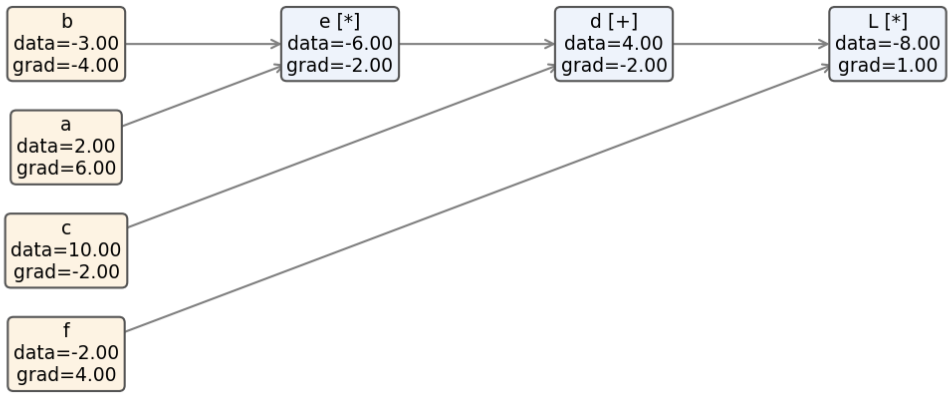


Figure 2: Graph for $L = (ab + c)f$ with $a = 2, b = -3, c = 10, f = -2$: forward values and the gradients from backprop.

5 Value Class Design

```
class Value:
    def __init__(self, data, _children=(), _op='', label=''):
        self.data = data                # scalar forward value
        self.grad = 0.0                 # dL/d(self), filled by backward()
        self._backward = lambda: None   # local chain-rule step (leaves: no-op)
        self._prev = set(_children)     # operands this node was computed from
        self._op = _op                  # operation label, e.g. '+', '*', 'tanh'
        self.label = label
```

- `_prev` is a *set*, so `a + a` stores `a` once. The gradient is still counted twice because the closure of the addition adds to `self.grad` and `other.grad`, which are the same object.
- `_backward` is a closure created inside each operator. It captures the operands and the output node, so it can apply the local derivative later without storing extra state.
- `__repr__` returns `Value(data=...)`. `__int__`/`__float__` convert the stored data. `reset_grad()` sets `grad` back to 0.

6 Operator Overloading

Every operator returns a new `Value` whose `_prev` contains its operands. Plain numbers are wrapped into constant `Values` by a small helper. Any other type makes the operator return `NotImplemented`, which Python turns into a `TypeError` (e.g. `Value(2.0) + "string"`):

```
@staticmethod
def _wrap(other):
    if isinstance(other, Value):
        return other
    if isinstance(other, (int, float)):
        return Value(other)
    return None          # caller returns NotImplemented -> TypeError

def __add__(self, other):
    other = Value._wrap(other)
    if other is None:
```

```
        return NotImplemented
    out = Value(self.data + other.data, (self, other), '+')
    def _backward():
        self.grad += 1.0 * out.grad
        other.grad += 1.0 * out.grad
    out._backward = _backward
    return out
```

Only $+$, $\times$, power, tanh and exp have their own backward rules. The other operators are *compositions*, so their gradients come out of the chain rule automatically:

Operator	Implementation
`-a` (`__neg__`)	`a * -1`
`a - b` (`__sub__`)	`a + (-b)`
`a / b` (`__truediv__`)	`a * b**-1`
`2 + a`, `2 * a`, `2 - a`, `2 / a`	`__radd__`, `__rmul__`, `__rsub__`, `__rtruediv__`

Division by zero raises `ZeroDivisionError` naturally, because 0^{-1} is evaluated by Python during the forward pass of `b**-1`. The exponent of `__pow__` must be an `int` or `float` (enforced with `assert`). A `Value` exponent is not supported.

7 Mathematical Derivations: Local Derivatives

Let L be the final scalar output and write $\bar{v} = \partial L/\partial v$ for the gradient stored in `v.grad`. Each `_backward` receives $\bar{o}$ of its output o and adds $\bar{o} \cdot \partial o/\partial x$ to each input x.

Addition $o = a + b$:

$$\frac{\partial o}{\partial a} = 1, \quad \frac{\partial o}{\partial b} = 1 \implies \bar{a} \mathrel{+}= \bar{o},\ \bar{b} \mathrel{+}= \bar{o}.$$

Multiplication $o = a\,b$:

$$\frac{\partial o}{\partial a} = b, \quad \frac{\partial o}{\partial b} = a \implies \bar{a} \mathrel{+}= b\,\bar{o},\ \bar{b} \mathrel{+}= a\,\bar{o}.$$

Power $o = a^n$, n a constant:

$$\frac{\partial o}{\partial a} = n\,a^{n-1} \implies \bar{a} \mathrel{+}= n\,a^{n-1}\,\bar{o}.$$

Exponential $o = e^a$:

$$\frac{\partial o}{\partial a} = e^a = o \implies \bar{a} \mathrel{+}= o\,\bar{o},$$

so the backward pass reuses the forward result `out.data`.

Hyperbolic tangent $o = \tanh a = \dfrac{e^a - e^{-a}}{e^a + e^{-a}}$. By the quotient rule,

$$\frac{d}{da}\tanh a = \frac{(e^a + e^{-a})^2 - (e^a - e^{-a})^2}{(e^a + e^{-a})^2} = 1 - \tanh^2 a \implies \bar{a} \mathrel{+}= (1 - o^2)\,\bar{o}.$$

Derived operators. For $o = a/b = a \cdot b^{-1}$ the engine gives $\bar{a} = \bar{o}/b$ and $\bar{b} = \bar{o} \cdot a \cdot (-1)b^{-2} = -\bar{o}\, a/b^2$, which are exactly the quotient-rule derivatives.

The corresponding code (from `src/engine.py`):

```
def __mul__(self, other):
    ...
    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad

def __pow__(self, other):
    assert isinstance(other, (int, float)), "only supporting int/float powers for now"
    out = Value(self.data ** other, (self,), f'**{other}')
    def _backward():
        self.grad += (other * self.data ** (other - 1)) * out.grad

def tanh(self):
    t = math.tanh(self.data)
    out = Value(t, (self,), 'tanh')
    def _backward():
        self.grad += (1 - t ** 2) * out.grad

def exp(self):
    e = math.exp(self.data)
    out = Value(e, (self,), 'exp')
    def _backward():
        self.grad += out.data * out.grad
```

8 The Chain Rule

If L depends on x through intermediate nodes, the multivariate chain rule states

$$\frac{\partial L}{\partial x} = \sum_{o \in \text{consumers}(x)} \frac{\partial L}{\partial o} \frac{\partial o}{\partial x}.$$

Two consequences shape the implementation:

1. **Product along a path.** Each `_backward` multiplies the incoming $\bar{o}$ by the local derivative, so gradients are products of local derivatives along each path from L to x.

2. **Sum over paths.** If x is used by several nodes, the contributions must be *added*. This is why every `_backward` uses `+=` rather than `=`.

Worked example (official test). With $a = -2$, $b = 3$, $d = ab$, $e = a + b$, $f = de$:

$$\frac{\partial f}{\partial a} = \underbrace{e \cdot b}_{\text{via } d} + \underbrace{d \cdot 1}_{\text{via } e} = (1)(3) + (-6) = -3, \qquad \frac{\partial f}{\partial b} = e \cdot a + d = (1)(-2) + (-6) = -8,$$

matching $\partial f/\partial a = 2ab + b^2$ and $\partial f/\partial b = a^2 + 2ab$. Our engine returns exactly -3 and -8.

9 Topological Sorting

A node's `_backward` may only run once its own gradient is complete, i.e. after *every* node that consumes it has run. Reversing a topological order (inputs before outputs) guarantees this.

```python
def _get_topo(self):
    topo = []
    visited = set()

    def build_topo(node):
        if node not in visited:
            visited.add(node)
            for child in node._prev:
                build_topo(child)
            topo.append(node)  # post-order: after all children

    build_topo(self)
    return topo
```

Correctness. Depth-first search with *post-order* insertion appends a node only after all of its inputs (and, recursively, their inputs) have been appended. So in `topo` every node appears after its inputs, and in `reversed(topo)` every node appears before its inputs. The DFS visits each node and edge once, so the cost is $O(V + E)$.

Answers to the TA warnings (common traps).

- *The post-order trap.* If a node were appended *before* visiting its children (pre-order), then for $f = d \cdot e$ with $d = ab$, $e = a + b$ the list could be $[f, d, a, b, e]$, and its reverse $[e, b, a, d, f]$ puts the root *last*. The seeded gradient $\bar{f} = 1$ would then be propagated only after e and d had already pushed their (still zero) gradients to a and b, so $\bar{a} = \bar{b} = 0$. Appending after the loop over children is what makes the order topological.
- *The diamond-graph trap.* In $c = a + a$, $d = c \cdot a$ the node a is reachable through several paths. Without `visited`, a (and in general whole sub-graphs) would be appended several times, so their `_backward` would run several times and gradients would be double-counted. The set ensures each node appears exactly once. Gradient accumulation (`+=`) still counts both uses of a, giving $\partial d/\partial a = 4a = 12$ for $a = 3$ (official test).

Limitation. The recursion follows the specified algorithm. Its depth equals the longest path in the graph, so graphs deeper than Python's recursion limit (≈ 1000) would need an iterative DFS. The graphs in this assignment (including training in the notebook) are far shallower.

10 Backpropagation Algorithm

```python
def backward(self):
    topo = self._get_topo()
    self.grad = 1.0               # seed: dL/dL = 1
    for node in reversed(topo):
        node._backward()
```

Seeding with 1 is the base case $\partial L/\partial L = 1$. Walking the reversed order, each node pushes its (now complete) gradient to its inputs. Leaves have a no-op `_backward`.

Gradient reset. Like PyTorch, gradients *accumulate* across calls: calling `backward()` twice doubles every leaf gradient. `backward()` therefore does not clear old gradients itself. The caller resets them with `reset_grad()` on a `Value`, or `zero_grad()` on a `Neuron`/`Layer`/`MLP`, before each new pass. This matches the specified algorithm and keeps accumulation available as a feature. The training loop calls `model.zero_grad()` every step.

11 Activation Functions

- tanh maps $\mathbb{R} \to (-1, 1)$, is zero-centred, and has derivative $1 - \tanh^2 x \in (0, 1]$ (maximal, $= 1$, at $x = 0$, as checked by the official test). For large $|x|$ it saturates and the derivative tends to 0 (vanishing gradients). Our extended test checks $x = 50$.
- exp is its own derivative. It overflows for large inputs. `Value(1000.0).exp()` raises `OverflowError` (from `math.exp`), which we test explicitly rather than returning `inf` silently.

Both are implemented as single graph nodes with closed-form derivatives. This is faster and numerically better than composing tanh from `exp`, additions and a division.

12 Neural Network Implementation

12.1 Neuron

```
class Neuron:
    def __init__(self, nin):
        self.w = [Value(random.uniform(-1, 1)) for _ in range(nin)]
        self.b = Value(random.uniform(-1, 1))

    def __call__(self, x):
        if isinstance(x, (Value, int, float)):   # width-1 layer output
            x = [x]
        if len(x) != len(self.w):
            raise ValueError(f"Neuron expects {len(self.w)} inputs, got {len(x)}")
        act = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)
        return act.tanh()

    def parameters(self):
        return self.w + [self.b]
```

A neuron computes $y = \tanh(\sum_{i=1}^{n} w_i x_i + b)$ with $w_i, b \sim \mathcal{U}(-1, 1)$. Random initialisation breaks the symmetry between neurons; if all weights were equal, all neurons of a layer would receive identical gradients and stay identical. The sum is started from `self.b` so the bias is included without an extra node. Inputs may be plain floats or `Value`s. Two small robustness additions: a single scalar input is accepted (needed when a hidden layer has width 1, see below), and a wrong input length raises `ValueError` instead of being silently truncated by `zip`.

12.2 Layer

```
class Layer:
    def __init__(self, nin, nout):
        self.neurons = [Neuron(nin) for _ in range(nout)]

    def __call__(self, x):
        outs = [n(x) for n in self.neurons]
        return outs[0] if len(outs) == 1 else outs

    def parameters(self):
        return [p for n in self.neurons for p in n.parameters()]
```

A layer is $\mathbf{y} = \tanh(W\mathbf{x} + \mathbf{b})$ computed neuron by neuron. It has $(n_{in} + 1)\, n_{out}$ parameters (e.g. 16 for `Layer(3, 4)`). As required by the template, a single-neuron layer returns a `Value` rather than a one-element list, so the output of a regression/binary network is directly a scalar.

12.3 MLP

```
class MLP:
    def __init__(self, nin, nouts):
        sizes = [nin] + list(nouts)
        self.layers = [Layer(sizes[i], sizes[i + 1]) for i in range(len(nouts))]

    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)
        return x
```

`MLP(3, [4, 4, 1])` has $(3{+}1)4 + (4{+}1)4 + (4{+}1)1 = 41$ parameters (official test). `zero_grad()` in every class calls `reset_grad()` on each parameter.

13 Testing Methodology

1. **Official suite** (`tests/test_engine.py`, `tests/test_nn.py`, unmodified): 12 tests covering representation, arithmetic, type errors, power, activations, diamond graphs, chain rule, division by zero, casting, and the three network classes.

2. **Extended suite** (`tests/test_extended.py`, written for this report, 30 tests).

   - *Numerical gradient checking*: each operator ($+, \times, -, /, a^3, a^{-2}, a^{0.5}, \exp, \tanh$, negation), a multi-level composite expression and a full `MLP(2, [3, 1])` loss are compared against central differences $\frac{f(x+h)-f(x-h)}{2h}$ with $h = 10^{-6}$ (tolerance 10^{-5}). For the MLP, all 13 parameter gradients are checked.
   - Scalars on both sides of every operator; repeated variables ($a \cdot a \cdot a$, $b + b + b + b$); a diamond with a shared intermediate.
   - Structural check of `_get_topo`: no duplicates, every node after its inputs, root last.
   - Accumulation over two `backward()` calls and reset; backward on a lone leaf; tanh saturation; `exp` overflow; unsupported types; division by a zero constant.
   - Networks: neuron output against a hand computation, wrong input size, five MLP configurations (including width-1 hidden layers and multi-output heads), `zero_grad` over all levels, and a check that gradient descent decreases the loss.

3. **Flakiness check**: `test_mlp` asserts that *no* parameter gradient is zero under unseeded initialisation. We re-ran its scenario for 5000 seeds, and no seed produced a zero gradient.

14 Testing Results

Command: `python -m pytest -v` (Python 3.11.9, pytest, Windows 11).

Test file	Tests	Passed
`tests/test_engine.py` (official)	9	9
`tests/test_nn.py` (official)	3	3
`tests/test_extended.py` (additional)	30	30
Total	**42**	**42**

Numerical gradient check from the notebook, at $(x_1, x_2, x_3) = (0.3, -1.2, 2.5)$, for

$$g = \left(\tanh(x_1 x_2 + x_3)\, e^{x_1^2} - x_2/x_3\right)^2 :$$

Input	Backprop	Finite difference	Abs. error
x_1	1.75384622	1.75384622	1.1×10^{-10}
x_2	−1.18082193	−1.18082193	2.6×10^{-10}
x_3	−0.41096164	−0.41096164	5.2×10^{-10}

The remaining error is the truncation/round-off error of the finite difference itself.

15 Notebook Demonstration

The notebook `notebooks/micrograd_demo.ipynb` walks through `Value` objects, every operation, graph construction and backprop (Fig. 2), gradient accumulation and gradient checking, the network classes, and finally an end-to-end training run. All numbers below come from executing it (random seed 42 for data, split and weight initialisation).

Dataset. scikit-learn `make_moons` (200 points, noise 0.1), labels mapped to $\{-1, +1\}$, stratified 75/25 train/test split (150/50). We chose it because it is not linearly separable, it is two-dimensional (so the decision boundary can be plotted), and it is small enough for a pure-Python scalar engine.

Model and training. `MLP(2, [16, 16, 1])` (337 parameters, all tanh). Loss: mean squared error between output and label plus L_2 penalty $10^{-4} \sum_j \theta_j^2$. Full-batch gradient descent for 100 steps, learning rate decayed linearly from 0.5 to 0.05. Each step runs the forward pass, the loss, `zero_grad()`, `backward()` and the update $\theta \leftarrow \theta - \eta\,\bar{\theta}$.

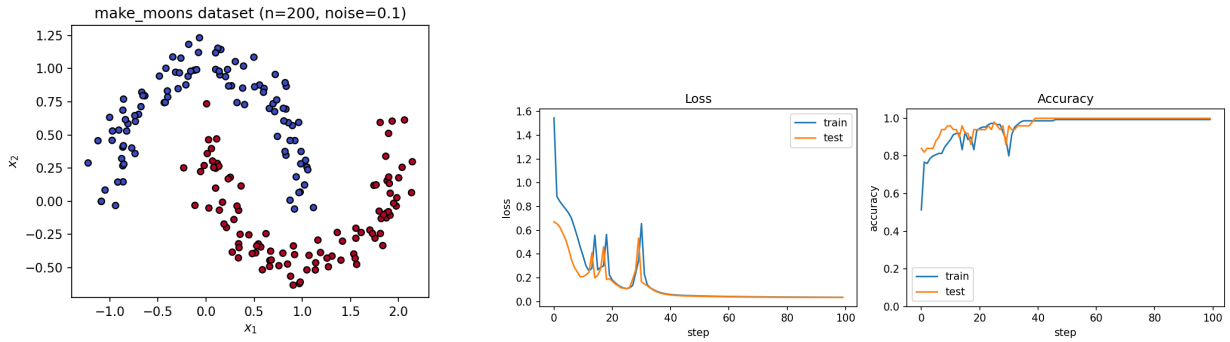


Figure 3: Left: dataset. Right: loss and accuracy per training step.

	Loss	Accuracy
Step 0 (train)	1.5459	51.3 %
Final (train, 150 points)	0.0353	99.3 %
Final (test, 50 points)	0.0341	100.0 %

Training took about 157 s on a laptop CPU. Around step 30 the training loss rose temporarily (0.18 at step 20 to 0.66 at step 30) before converging. This is an overshoot caused by the large early learning rate, and it disappears as the rate decays. With only 50 test points, 100 % test accuracy means the boundary separates this particular split. It should not be read as a precise error estimate.

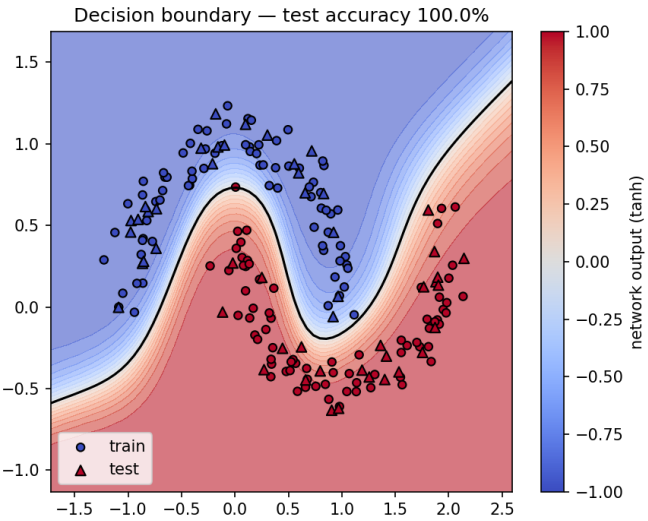


Figure 4: Learned decision boundary (black contour = output 0). Circles: train, triangles: test.

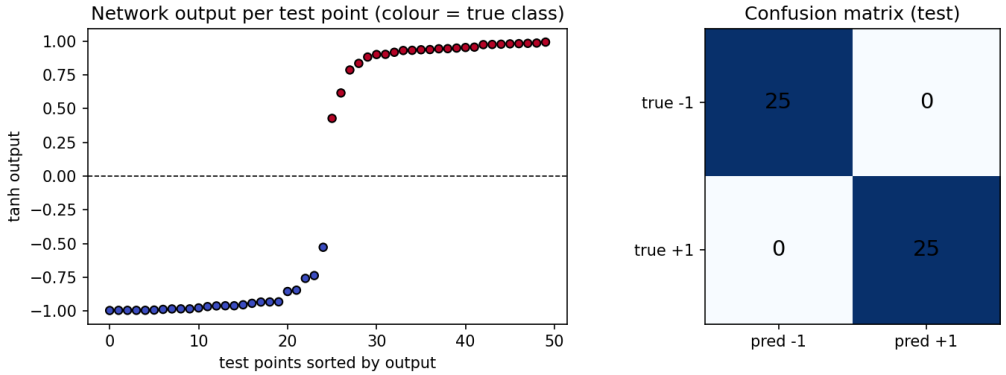


Figure 5: Test-set outputs coloured by true class, and the test confusion matrix (25/25 correct per class).

16 Design Justifications

- **Closures for `_backward`.** Each node carries exactly the information its derivative needs. There is no central table of operations, and adding an operator only means writing one method.
- **Minimal primitive set.** Only $+$, $\times$, power, tanh, exp have hand-written derivatives. Subtraction, negation and division are compositions, so they cannot have inconsistent gradients.
- **`NotImplemented` for unsupported types** instead of raising directly. Python then tries the reflected operator of the other operand and finally raises the standard `TypeError`, which is the idiomatic protocol.
- **`assert` in `__pow__`** as required by the template. A `Value` exponent would need the derivative $a^n \ln a$ with respect to n, which is out of scope.
- **Accumulating gradients** (`+=`, no automatic reset) for correctness on shared nodes and to match the PyTorch convention, with explicit `zero_grad()`.

- **Recursive DFS** exactly as specified. It is simple and $O(V + E)$. Its limitation (recursion depth) is documented above.
- **Template API preserved**: all class and method signatures are unchanged, so the official autograder runs as-is. The only additions are `__rsub__`, `__rtruediv__` (so `1 - x` and `1 / x` work), the `_wrap` helper and the input checks in `Neuron`.

17 Conclusion

We implemented a complete scalar reverse-mode automatic differentiation engine and a small neural-network library on top of it. The engine builds the computational graph through operator overloading, attaches the local derivative of each operation as a closure, orders the graph with a post-order DFS, and backpropagates by applying the chain rule in reverse topological order with accumulated gradients. All 12 official tests and 30 additional tests pass, and analytic gradients agree with finite differences to about 10^{-10}. Using only this engine, an MLP trained by gradient descent learns the non-linear two-moons boundary (99.3 % train, 100 % test accuracy). The main limitation is speed: one Python object per scalar. Frameworks such as PyTorch apply the same algorithm to tensors.